SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL (SENAI)

PROGRAMAÇÃO ORIENTADA A OBJETOS

MAKGREGOR ALVES ALTÊ

ATIVIDADE 01
CADASTRO DE CLIENTES DA ELECTRONIC:
DA MODELAGEM AO CÓDIGO

RIO GRANDE - RS

2026

1 ANÁLISE DO PROBLEMA

A Electronic necessita de um módulo para registrar clientes do comércio eletrônico. O sistema deve aceitar pessoas físicas e jurídicas, preservando os dados comuns em uma classe base e os dados específicos em classes derivadas. O cadastro contém nome e endereço, além de CPF e data de nascimento para pessoas físicas, ou CNPJ e razão social para pessoas jurídicas.

O módulo mantém uma lista de clientes e só insere nela os objetos construídos com dados válidos. Quando ocorre uma entrada não permitida, uma exceção apresenta o motivo da rejeição e o processamento continua, sem afetar os demais registros.

As regras aplicadas são: nome com no mínimo três caracteres; CPF com 11 dígitos; CNPJ com 14 dígitos; remoção da pontuação dos documentos; data de nascimento válida e não futura; e razão social obrigatória. Também são conferidos os campos do endereço: logradouro, número, cidade e UF.

2 MODELAGEM ORIENTADA A OBJETOS

A classe Pessoa concentra nome e endereço, enquanto PessoaFisica e PessoaJuridica especializam esses dados utilizando herança. Endereco representa o conjunto de informações de localização associado a cada cliente. O diagrama apresenta os atributos, os métodos, a visibilidade e as relações entre as quatro classes.

O sinal “-” identifica membros privados, “+” identifica membros públicos e “#” indica comportamento destinado às subclasses. Como JavaScript não dispõe de protected, o método _resumoBase() utiliza a convenção de prefixo sublinhado.

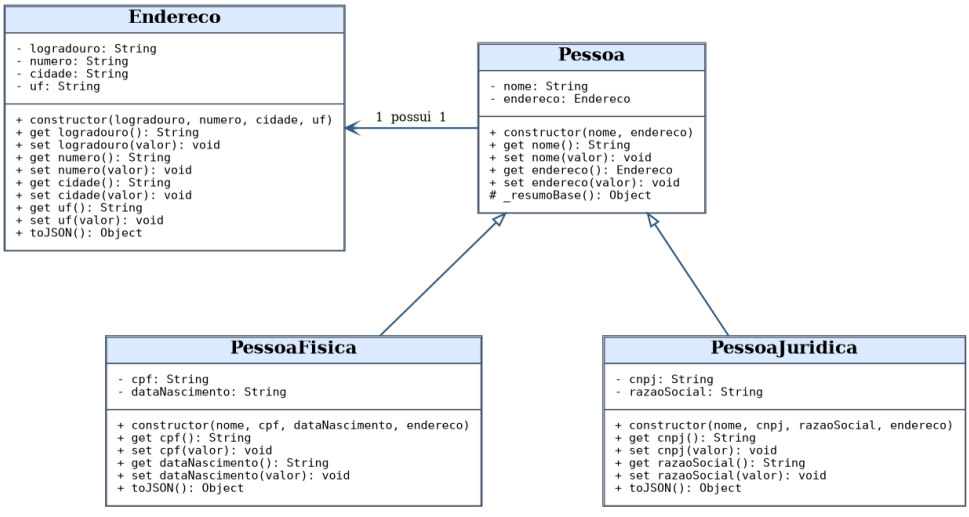


Figura 1 - Diagrama de classes. Fonte: elaboração própria (2026).

3 IMPLEMENTAÇÃO EM JAVASCRIPT

As classes foram implementadas em arquivos independentes, no diretório src. Os construtores chamam os setters, responsáveis pela validação dos atributos, que são protegidos por campos privados (#). Os getters permitem a leitura dos valores armazenados sem expor diretamente os campos internos.

As duas classes derivadas utilizam extends para herdar de Pessoa e super(nome, endereco) para inicializar os atributos compartilhados. Os métodos toJSON() organizam os dados apresentados na listagem final. A função cadastrar(), definida em index.js, captura as exceções com try/catch e evita a interrupção do programa quando um cadastro é recusado.

A validação de CPF e CNPJ verifica a quantidade exigida de dígitos após remover a pontuação, conforme definido na atividade. A data de nascimento é validada quanto ao formato, à existência no calendário e à ocorrência de datas futuras.

4 RESULTADOS DA EXECUÇÃO

A aplicação foi executada com o comando node index.js. Foram considerados quatro casos: pessoa física válida, pessoa jurídica válida, pessoa física com CPF incompleto e pessoa jurídica sem razão social. Os resultados apresentados no terminal foram:

Cenário 1 - Pessoa física válida: CADASTRADO
Cenário 2 - Pessoa jurídica válida: CADASTRADO
Cenário 3 - Pessoa física inválida (CPF): ERRO - CPF deve conter exatamente 11 dígitos.
Cenário 4 - Pessoa jurídica inválida (razão social): ERRO - Razão social é obrigatória.

Ao final da execução, a lista continha os dois registros válidos, com seus respectivos endereços, enquanto as duas entradas inválidas foram rejeitadas. A saída integral do terminal está reproduzida no README.md, na seção de evidências de execução.